



Universidad Autónoma de Baja California

FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA



Programación orientada a objetos.

Práctica 2: Inicialización de objetos precisando los atributos e invocación de métodos.

Alumno:

Joshua Osorio O. – 1293271

Docente:

Itzel Barriba Cázares

Marzo/2026

Tabla de contenido

Objetivo.....	3
Lista de materiales	3
Descripción del taller	3
DESARROLLO DE LA PRACTICA.	5
Diagrama UML	5
Clase Principal:	5
Clase Libro.	9
Clase Empleado.	12
Análisis de resultados	16
Programa 1 – Libro.....	16
Programa 2 – Empleado con un nombre	17
Conclusiones	18
Criterios para evaluar:.....	18
Código (70 puntos).....	18
Reporte (30 puntos):.....	18

Objetivo

El alumno diseñará clases para modelar el estado y comportamiento de entidades del mundo real, a través de atributos y métodos, respectivamente, respetando los principios de la orientación a objetos y las capacidades del lenguaje de programación Java, con una actitud crítica, propositiva y creativa.

En esta práctica se implementaron dos programas:

- Programa 1: una clase Libro que modela un libro en una biblioteca, con sus atributos y un método para imprimir su información.
- Programa 2: una clase Empleado con nombre completo (considerando uno o dos nombres), fecha de nacimiento y salario semanal, incluyendo un método para calcular el salario mensual.

Lista de materiales

- Computadora Personal (PC)
- Ambiente de desarrollo.
- JDK Java Development Kit

Descripción del taller

El alumno diseñará e implementará una clase que modele una Cuenta Bancaria, aplicando correctamente los principios de POO, específicamente:

- Encapsulamiento
- Uso de constructores
- Sobrecarga de constructores
- Sobrecarga de métodos
- Validación de datos
- Separación entre lógica y clase principal.

Diseño UML

Antes de programar, realiza el modelado UML de la clase cuenta

Determina los atributos de la clase, e implementa los métodos para encapsulamiento, depositar(), retirar(), mostrarDatos() y constructores.

El diagrama debe incluir:

- Visibilidad
- Tipo de datos
- Parámetros

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

- Tipo de retorno

Implementar constructores:

- Constructor por defecto (inicializa el saldo en 0),
- Constructor completo (inicializa todos los atributos)
- Constructor sobrecargado (Nombre completo, fecha de nacimiento, saldo inicial)

Manejo de métodos:

Debe existir un método depositar sobrecargado (cantidad) y (cantidad y concepto). No debe permitir

cantidades negativas, mostrar el saldo anterior y el saldo actualizado, si existe concepto, mostrarlo.

El método retirar sobrecargado (cantidad) y (cantidad y concepto). No debe permitir cantidades negativas, no permitir retirar más del saldo disponible. Mostrar mensaje de fondos insuficiente si aplica, mostrar el saldo anterior y el saldo actualizado.

El método Mostrar datos debe devolver un String con nombre completo, dirección completa, fecha de nacimiento, saldo actual.

La clase principal

Crear una clase Principal que:

- Capture los datos mediante menú.
- Solo realice llamadas a métodos
- No contenga lógica bancaria
- Permita repetir operaciones hasta que el usuario seleccione salir.

Menú de Opciones

1. Modificar Nombre
2. Modificar Apellido Paterno
3. Modificar Apellido Materno
4. Modificar Calle
5. Modificar Número
6. Modificar Colonia
7. Modificar Código Postal
8. Modificar Fecha de Nacimiento
9. Depositar
10. Retirar
11. Mostrar datos completos
12. Salir

Cuando se modifique un atributo:

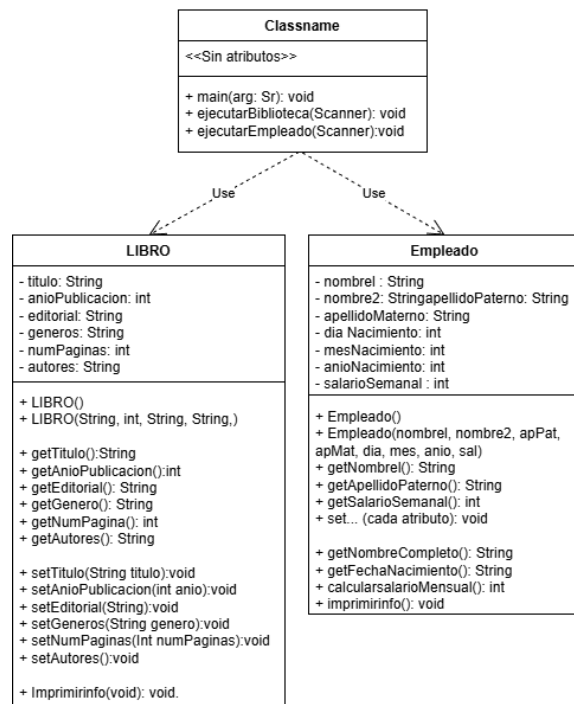
- Mostrar el valor anterior
- Mostrar el nuevo valor.

Adjunte el diagrama de clase y el código fuente, también deberá hacer una demostración de su ejecución en la clase de laboratorio.

DESARROLLO DE LA PRACTICA.

Diagrama UML

A continuación, se presenta el diagrama de clases UML que modela la solución. La clase Principal depende de Libro y Empleado mediante una relación de uso («uses»).



Clase Principal:

La clase Principal actúa como punto de entrada del programa y como interfaz de usuario. No contiene lógica de negocio; su única responsabilidad es presentar un menú, capturar datos del usuario e invocar los métodos correspondientes.

El método main() implementa un ciclo do-while con un switch de dos opciones:

- Opción 1: llama al método estático ejecutarBiblioteca(input).
- Opción 2: llama al método estático ejecutarEmpleado(input).
- Opción 0: termina el programa.

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

El método ejecutarBiblioteca(Scanner input) captura los seis atributos del libro mediante `input.nextLine()` para cadenas e `input.nextInt()` para enteros, con `input.nextLine()` después de cada `nextInt()` para limpiar el buffer. Al final crea un objeto Libro con el constructor completo y llama a `libro.imprimirInfo()`.

El método ejecutarEmpleado(Scanner input) primero pregunta al usuario si tiene segundo nombre. Si responde "S", captura `nombre1` y `nombre2`; si responde "N", solo captura `nombre1` y deja `nombre2` como cadena vacía. Luego captura los apellidos, los datos de nacimiento y el salario semanal. Finalmente crea un objeto Empleado con el constructor completo y llama a `empleado.imprimirInfo()`.

```

/*-----
Practica 2: Inicialización de objetos precisando los atributos e invocación de métodos.
Nombre: Joshua Osorio
Materia: Programación Orientada a Objetos
Fecha: Febrero/2026
-----*/

import java.util.Scanner;

public class Principal {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int opcion;

        do {
            System.out.flush();
            System.out.printf("\n\n\tPráctica 2 - POO\n");
            System.out.println("\n\t----- Menú -----");
            System.out.println("\t1 - Programa 1: Biblioteca (Libro)");
            System.out.println("\t2 - Programa 2: Empleado");
            System.out.println("\t0 - Salir");
            System.out.printf("\tOpción:\t");
            opcion = input.nextInt();
            input.nextLine();

            switch (opcion) {
                case 1:
                    ejecutarBiblioteca(input);
                    break;
                case 2:
                    ejecutarEmpleado(input);
                    break;
                case 0:
                    System.out.printf("\n\tSaliendo...\n");
                    break;
                default:
                    System.out.printf("\n\tOpción no válida, intente nuevamente.\n");
                    break;
            }

        } while (opcion != 0);

        input.close();
    }
}

```

```
// -----  
// Programa 1: captura datos de un Libro y los imprime.  
// -----  
public static void ejecutarBiblioteca(Scanner input) {  
    System.out.println("\n\n\t--- Programa 1: Registro de Libro ---");  
  
    System.out.printf("\tTítulo:\t\t\t");  
    String titulo = input.nextLine();  
  
    System.out.printf("\tAño de publicación:\t");  
    int anio = input.nextInt();  
    input.nextLine();  
  
    System.out.printf("\tEditorial:\t\t");  
    String editorial = input.nextLine();  
  
    System.out.printf("\tGéneros:\t\t");  
    String generos = input.nextLine();  
  
    System.out.printf("\tNúmero de páginas:\t");  
    int paginas = input.nextInt();  
    input.nextLine();  
  
    System.out.printf("\tAutores:\t\t");  
    String autores = input.nextLine();  
  
    // Crear el objeto Libro con los datos capturados.  
    Libro libro = new Libro(titulo, anio, editorial, generos, paginas, autores);  
  
    // Imprimir la información del libro.  
    System.out.println();  
    libro.imprimirInfo();  
}
```



```

// -----
// Programa 2: captura datos de un Empleado y los imprime.
// -----
public static void ejecutarEmpleado(Scanner input) {
    System.out.println("\n\n\t--- Programa 2: Registro de Empleado ---");

    // Captura del nombre - considera 1 o 2 nombres.
    System.out.printf("\t¿Tiene segundo nombre? S / N:\t");
    String resp = input.nextLine();

    String nombre1 = "", nombre2 = "";
    System.out.printf("\tPrimer nombre:\t\t");
    nombre1 = input.nextLine();

    if (resp.equalsIgnoreCase("S")) {
        System.out.printf("\tSegundo nombre:\t\t");
        nombre2 = input.nextLine();
    }

    System.out.printf("\tApellido paterno:\t");
    String apellidoPaterno = input.nextLine();

    System.out.printf("\tApellido materno:\t");
    String apellidoMaterno = input.nextLine();

    // Captura de fecha de nacimiento.
    System.out.printf("\tDía de nacimiento:\t");
    int dia = input.nextInt();

    System.out.printf("\tMes de nacimiento:\t");
    int mes = input.nextInt();

    System.out.printf("\tAño de nacimiento:\t");
    int anio = input.nextInt();

    // Captura del salario semanal.
    System.out.printf("\tSalario semanal ($):\t");
    int salario = input.nextInt();
    input.nextLine();

    // Crear el objeto Empleado con los datos capturados.
    Empleado empleado = new Empleado(nombre1, nombre2, apellidoPaterno, apellidoMaterno,
        dia, mes, anio, salario);

    // Imprimir la información y el salario mensual.
    System.out.println();
    empleado.imprimirInfo();
}
}

```

Clase Libro.

La clase Libro representa un libro dentro de una biblioteca. Sus seis atributos son privados, aplicando encapsulamiento:

- titulo (String): título del libro.

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

- anioPublicacion (int): año en que fue publicado.
- editorial (String): nombre de la editorial.
- generos (String): género o géneros literarios del libro.
- numPaginas (int): número total de páginas.
- autores (String): nombre o nombres de los autores.

Se implementaron dos constructores: uno por defecto que inicializa todos los atributos de texto con "No capturado" y los numéricos con 0, y uno completo que recibe los seis atributos como parámetros para inicializar el objeto en un solo paso.

Para cada atributo se proporcionan un getter (método get) y un setter (método set), siguiendo las convenciones de nomenclatura de Java. El método imprimirInfo() imprime en consola todos los atributos del libro con etiquetas descriptivas usando System.out.println.

```
/*-----  
Practica 2: Inicialización de objetos precisando los atributos e invocación de métodos.  
Nombre: Joshua Osorio  
Materia: Programación Orientada a Objetos  
Fecha: Febrero/2026  
-----*/  
  
public class Libro {  
  
    // Atributos privados de la clase.  
    private String titulo;  
    private int anioPublicacion;  
    private String editorial;  
    private String generos;  
    private int numPaginas;  
    private String autores;  
  
    // Constructor por defecto.  
    public Libro() {  
        this.titulo = "No capturado";  
        this.anioPublicacion = 0;  
        this.editorial = "No capturado";  
        this.generos = "No capturado";  
        this.numPaginas = 0;  
        this.autores = "No capturado";  
    }  
  
    // Constructor completo.  
    public Libro(String titulo, int anioPublicacion, String editorial,  
        String generos, int numPaginas, String autores) {  
        this.titulo = titulo;  
        this.anioPublicacion = anioPublicacion;  
        this.editorial = editorial;  
        this.generos = generos;  
        this.numPaginas = numPaginas;  
        this.autores = autores;  
    }  
  
    // Getters.  
    public String getTitulo() {  
        return titulo;  
    }  
  
    public int getAnioPublicacion() {  
        return anioPublicacion;  
    }  
  
    public String getEditorial() {  
        return editorial;  
    }  
  
    public String getGeneros() {  
        return generos;  
    }  
}
```

```

public int getNumPaginas() {
    return numPaginas;
}

public String getAutores() {
    return autores;
}

// Setters.
public void setTitulo(String titulo) {
    this.titulo = titulo;
}

public void setAnioPublicacion(int anioPublicacion) {
    this.anioPublicacion = anioPublicacion;
}

public void setEditorial(String editorial) {
    this.editorial = editorial;
}

public void setGeneros(String generos) {
    this.generos = generos;
}

public void setNumPaginas(int numPaginas) {
    this.numPaginas = numPaginas;
}

public void setAutores(String autores) {
    this.autores = autores;
}

// Método para imprimir todos los atributos del libro.
public void imprimirInfo() {
    System.out.println("--- Información del Libro ---");
    System.out.println("Título:\t\t\t" + titulo);
    System.out.println("Año de publicación:\t" + anioPublicacion);
    System.out.println("Editorial:\t\t" + editorial);
    System.out.println("Géneros:\t\t" + generos);
    System.out.println("Número de páginas:\t" + numPaginas);
    System.out.println("Autores:\t\t" + autores);
}
}

```

Clase Empleado.

La clase Empleado modela a un trabajador con su información personal y salarial. Sus atributos privados son:

- nombre1 (String): primer nombre del empleado.
- nombre2 (String): segundo nombre, puede quedar vacío si el empleado tiene solo un nombre.

UABC_FCQI_PROGRAMACIÓNORIENTAOBJETOS.

- apellidoPaterno (String): apellido paterno.
- apellidoMaterno (String): apellido materno.
- diaNacimiento, mesNacimiento, anioNacimiento (int): componentes de la fecha de nacimiento almacenados por separado.
- salarioSemanal (int): salario que percibe el empleado por semana.

Se implementaron dos constructores: uno por defecto con valores iniciales predeterminados y uno completo que recibe todos los atributos. Los getters y setters permiten acceder y modificar cada atributo de forma controlada.

El método getNombreCompleto() concatena los componentes del nombre. Primero agrega nombre1 con un espacio; si nombre2 no está vacío usando (!nombre2.isEmpty()), lo agrega también; finalmente concatena apellidoPaterno y apellidoMaterno. Esto resuelve el requerimiento de considerar empleados con uno o dos nombres.

El método getFechaNacimiento() devuelve la fecha en formato dd/mm/aaaa concatenando los tres enteros con el separador "/".

El método calcularSalarioMensual() multiplica el salarioSemanal por 4 y devuelve el resultado como entero, asumiendo cuatro semanas por mes.

El método imprimirInfo() imprime en consola el nombre completo (llamando a getNombreCompleto()), la fecha de nacimiento (llamando a getFechaNacimiento()), el salario semanal y el salario mensual (llamando a calcularSalarioMensual()).

```

/*-----
Practica 2: Inicialización de objetos precisando los atributos e invocación de métodos.
Nombre: Joshua Osorio
Materia: Programación Orientada a Objetos
Fecha: Febrero/2026
-----*/

public class Empleado {

    // Atributos privados de la clase.
    private String nombre1;
    private String nombre2;
    private String apellidoPaterno;
    private String apellidoMaterno;
    private int diaNacimiento;
    private int mesNacimiento;
    private int anioNacimiento;
    private int salarioSemanal;

    // Constructor por defecto.
    public Empleado() {
        this.nombre1 = "No capturado";
        this.nombre2 = "";
        this.apellidoPaterno = "No capturado";
        this.apellidoMaterno = "No capturado";
        this.diaNacimiento = 1;
        this.mesNacimiento = 1;
        this.anioNacimiento = 2000;
        this.salarioSemanal = 500;
    }

    // Constructor completo.
    public Empleado(String nombre1, String nombre2, String apellidoPaterno, String apellidoMa-
terno,
        int diaNacimiento, int mesNacimiento, int anioNacimiento, int salarioSemanal) {
        this.nombre1 = nombre1;
        this.nombre2 = nombre2;
        this.apellidoPaterno = apellidoPaterno;
        this.apellidoMaterno = apellidoMaterno;
        this.diaNacimiento = diaNacimiento;
        this.mesNacimiento = mesNacimiento;
        this.anioNacimiento = anioNacimiento;
        this.salarioSemanal = salarioSemanal;
    }

    // Getters.
    public String getNombre1() {
        return nombre1;
    }

    public String getNombre2() {
        return nombre2;
    }
}

```

```
public String getApellidoPaterno() {
    return apellidoPaterno;
}

public String getApellidoMaterno() {
    return apellidoMaterno;
}

public int getDiaNacimiento() {
    return diaNacimiento;
}

public int getMesNacimiento() {
    return mesNacimiento;
}

public int getAnioNacimiento() {
    return anioNacimiento;
}

public int getSalarioSemanal() {
    return salarioSemanal;
}

// Setters.
public void setNombre1(String nombre1) {
    this.nombre1 = nombre1;
}

public void setNombre2(String nombre2) {
    this.nombre2 = nombre2;
}

public void setApellidoPaterno(String apellidoPaterno) {
    this.apellidoPaterno = apellidoPaterno;
}

public void setApellidoMaterno(String apellidoMaterno) {
    this.apellidoMaterno = apellidoMaterno;
}

public void setDiaNacimiento(int diaNacimiento) {
    this.diaNacimiento = diaNacimiento;
}

public void setMesNacimiento(int mesNacimiento) {
    this.mesNacimiento = mesNacimiento;
}

public void setAnioNacimiento(int anioNacimiento) {
    this.anioNacimiento = anioNacimiento;
}
```

```

// Devuelve el nombre completo concatenado.
public String getNombreCompleto() {
    String nombreCompleto = nombre1 + " ";
    if (!nombre2.isEmpty()) {
        nombreCompleto += nombre2 + " ";
    }
    nombreCompleto += apellidoPaterno + " " + apellidoMaterno;
    return nombreCompleto;
}

// Devuelve la fecha de nacimiento en formato dd/mm/aaaa.
public String getFechaNacimiento() {
    return diaNacimiento + "/" + mesNacimiento + "/" + anioNacimiento;
}

// Calcula el salario mensual (4 semanas).
public int calcularSalarioMensual() {
    return salarioSemanal * 4;
}

// Método para imprimir los datos del empleado.
public void imprimirInfo() {
    System.out.println("--- Información del Empleado ---");
    System.out.println("Nombre completo:\t" + getNombreCompleto());
    System.out.println("Fecha de nacimiento:\t" + getFechaNacimiento());
    System.out.println("Salario semanal:\t$" + salarioSemanal);
    System.out.println("Salario mensual:\t$" + calcularSalarioMensual());
}
}

```

Análisis de resultados

Programa 1 – Libro

Al seleccionar la opción 1 del menú, el programa solicita los seis atributos del libro de forma secuencial. Una vez ingresados todos los datos, el método `imprimirInfo()` los muestra en consola con sus etiquetas correspondientes. Por ejemplo, para un libro como "El nombre del viento", año 2007, editorial Debolsillo, género Fantasía, 662 páginas y autor Patrick Rothfuss, la salida mostraría:


```

Práctica 2 - POO

----- Menú -----

1 - Programa 1: Biblioteca (Libro)
2 - Programa 2: Empleado
0 - Salir

Opción: 1

----- Programa 1: Registro de Libro -----
Título:          Staring Out With Java
Año de publicación: 2016
Editorial:       Kelsey Loanes
Géneros:         Educacion y tecnologia
Número de páginas: 1225
Autores:         Tony Gaddis

----- Información del Libro -----
Título:          Staring Out With Java
Año de publicación: 2016
Editorial:       Kelsey Loanes
Géneros:         Educacion y tecnologia
Número de páginas: 1225
Autores:         Tony Gaddis

Práctica 2 - POO

----- Menú -----

1 - Programa 1: Biblioteca (Libro)
2 - Programa 2: Empleado
0 - Salir

Opción: |

```

Esto confirma que el constructor completo inicializa correctamente todos los atributos y que el método imprimirInfo() los presenta de forma ordenada y legible.

Programa 2 – Empleado con un nombre

Cuando el usuario indica que sí tiene segundo nombre (respuesta "S"), el método getNombreCompleto() incluye nombre2 en la concatenación. Para un empleado llamado Joe Isac Osorio Bernal, la salida mostraría el nombre completo con ambos nombres correctamente concatenados, confirmando que el manejo del caso de uno o dos nombres funciona según lo requerido.

```

Práctica 2 - POO

----- Menú -----

1 - Programa 1: Biblioteca (Libro)
2 - Programa 2: Empleado
0 - Salir

Opción: 2

----- Programa 2: Registro de Empleado -----
¿Tiene segundo nombre? S / N: S
Primer nombre:  Joshua
Segundo nombre:  Oleg
Apellido paterno: Osorio
Apellido materno: Bernal
Día de nacimiento: 11
Mes de nacimiento: 06
Año de nacimiento: 2002
Salario semanal ($): 1500

----- Información del Empleado -----
Nombre completo:  Joshua Oleg Osorio Bernal
Fecha de nacimiento: 11/6/2002
Salario semanal:  $1500
Salario mensual:  $6000

Práctica 2 - POO

----- Menú -----

1 - Programa 1: Biblioteca (Libro)
2 - Programa 2: Empleado
0 - Salir

Opción: |

```

```

Práctica 2 - POO
----- Menú -----
1 - Programa 1: Biblioteca (Libro)
2 - Programa 2: Empleado
0 - Salir

Opción: 0

Saliendo ...
Okuyt ~\OneDrive - uabc.edu.mx\8_Semestre\36282_ProgramaciónOrientadaAObjetos\2_Laboratorio\L02_Tema main ==*710 ~11 -7 1m 10.982s
>

```

Conclusiones

La práctica permitió comprender de forma práctica y aplicada los principios fundamentales de la Programación Orientada a Objetos. A través del modelado de una cuenta bancaria, se consolidaron los siguientes aprendizajes: Encapsulamiento, sobre carga de constructores, sobrecarga de métodos, validación de datos y separación de responsabilidades.

Criterios para evaluar:

Código (70 puntos)

- Debe incluir comentarios en su programa, sin llegar al uso exagerado de ellos.
- Debe utilizar sangrías en los lugares apropiados.
- Poner como encabezado del programa en comentarios, título, la descripción de la práctica, nombre del alumno que realizó la práctica.
- Funcionamiento correcto del programa

Reporte (30 puntos):

Portada, competencia u objetivo	2.5 puntos
Pegar Código del programa (anexar archivos .py)	2.5 puntos
Explicación completa del código y su funcionamiento	15 puntos
Resultados (análisis de resultados)	5 puntos
Conclusiones:	5 puntos

Nota: No se aceptarán trabajos fuera de la fecha de entrega o que no cumplan con las especificaciones de entrega requeridas. Los trabajos que se detecten copiados serán anulados automáticamente para ambos alumnos.